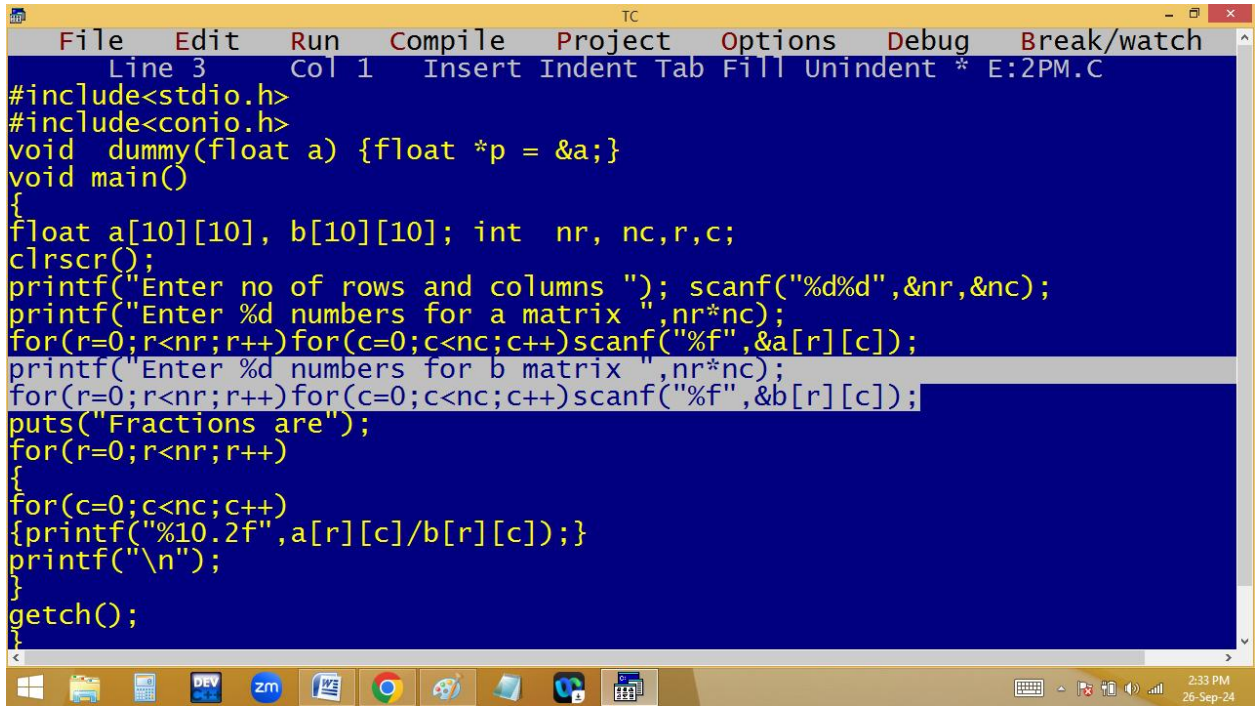


Finding fractions of $n \times n$ matrix:



```
File Edit Run Compile Project Options Debug Break/watch
Line 3 Col 1 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void dummy(float a) {float *p = &a;}
void main()
{
float a[10][10], b[10][10]; int nr, nc, r, c;
clrscr();
printf("Enter no of rows and columns "); scanf("%d%d", &nr, &nc);
printf("Enter %d numbers for a matrix ", nr*nc);
for(r=0; r<nr; r++) for(c=0; c<nc; c++) scanf("%f", &a[r][c]);
printf("Enter %d numbers for b matrix ", nr*nc);
for(r=0; r<nr; r++) for(c=0; c<nc; c++) scanf("%f", &b[r][c]);
puts("Fractions are");
for(r=0; r<nr; r++)
{
for(c=0; c<nc; c++)
{printf("%10.2f", a[r][c]/b[r][c]);}
printf("\n");
}
getch();
}
```

The screenshot shows a Turbo C++ IDE window titled "TC". The menu bar includes File, Edit, Run, Compile, Project, Options, Debug, and Break/watch. The status bar at the bottom indicates "Line 3 Col 1" and "Insert Indent Tab Fill Unindent * E:2PM.C". The code is written in C and defines two 10x10 matrices, a and b. It prompts the user to enter the number of rows and columns, then the elements of matrix a, and finally the elements of matrix b. After printing "Fractions are", it calculates the fraction of each element in matrix a relative to the corresponding element in matrix b (a[r][c]/b[r][c]) and prints it with a precision of 10.2f. The program uses clrscr() to clear the screen and getch() to wait for a key press before exiting.

```

Enter no of rows and columns 2 2
Enter 4 numbers for a matrix 6 2 4 5
Enter 4 numbers for b matrix 5 4 6 2
Fractions are
    1.20      0.50
    0.67      2.50

```

a

6 0,0	2 0,1
4 1,0	5 1,1

/

b

5 0,0	4 0,1
6 1,0	2 1,1

=

1.20.5

0.672.5

Matrix multiplication:

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 16 Col 2 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h> #include<conio.h>
void main()
{
int a[10][10], b[10][10], nr, nc, r, c, k, s; clrscr();
printf("Enter no of rows and columns "); scanf("%d%d",&nr,&nc);
printf("Enter %d numbers for a matrix ",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&a[r][c]);
printf("Enter %d numbers for b matrix ",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&b[r][c]);
puts("Result elements are");
for(r=0;r<nr;r++)
{
for(c=0;c<nc;c++)
{for(k=s=0;k<nc;k++)
{s=s+a[r][k]*b[k][c];}
printf("%4d",s);
}
printf("\n");
}
getch();
}
```

```

Enter no of rows and columns 2 2
Enter 4 numbers for a matrix 1 2 3 4
Enter 4 numbers for b matrix 5 6 7 8
Result elements are
19 22
43 50

```

```

for( r=0; r<2; r++ )
{
for( c=0; c<2; c++ )
{
for( s=k=0; k<2; k++ )
{
s=s+a[r][k]*b[k][c];
}
p(s);
}
p("\n");
}

```

r	c	k	s
0	0	012	0+1*5=5+2*7=19
0	1	012	0+1*6=6+2*8=22
1	0	012	0+3*5=15+4*7=43
1	1	012	0+3*6=18+4*8=50

a		b	
1	2	5	6
3	4	7	8

$1*5+2*7=19$ ✓ $1*6+2*8=22$ ✓
 $3*5+4*7=43$ ✓ $3*6+4*8=50$ ✓

3-dimensional arrays:

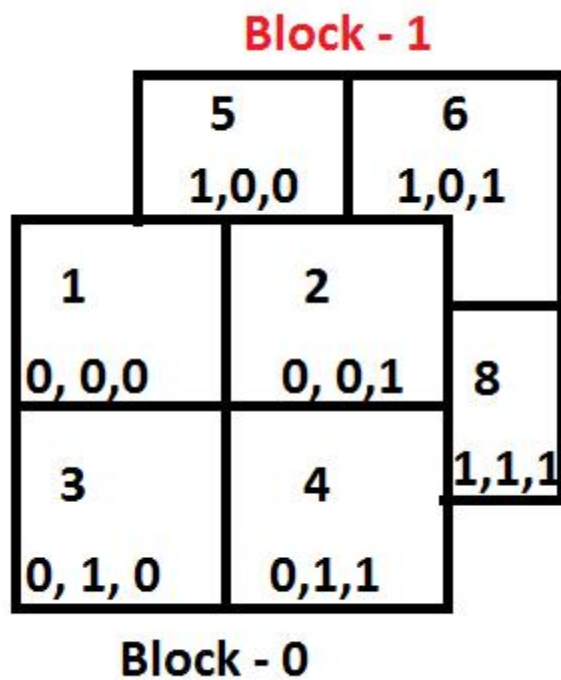
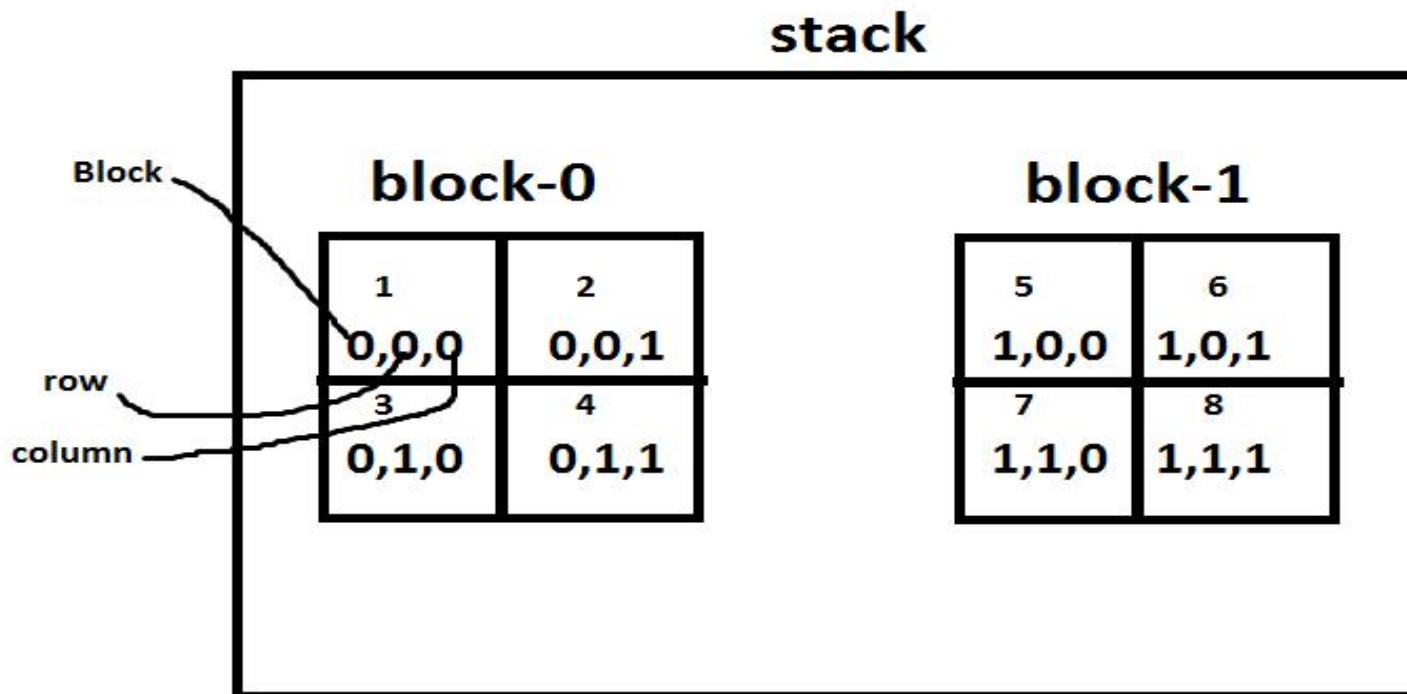
An array with several blocks, rows and columns.

An array with 3 subscripting operators **[] [] []**.

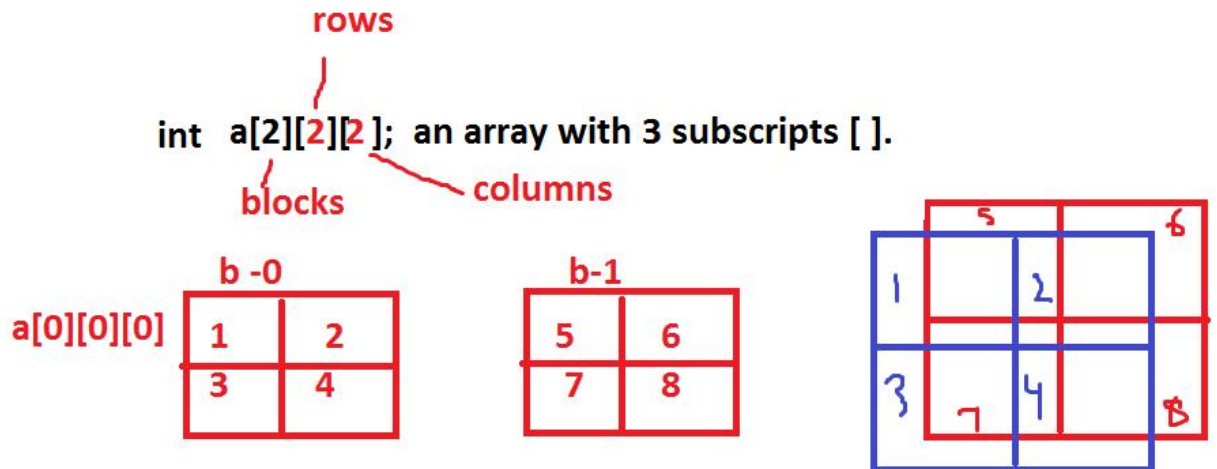
Syntax:

datatype variable [blocks] [rows] [columns];

Eg: `int a[2][2][2]={1,2,3,4,5,6,7,8};`



Eg:



eg: int class[2][60][6];
datatype class[sections][stus][marks];

```

TC
File Edit Run Compile Project Options Debug Break/
Edit
Line 1 Col 2 Insert Indent Tab Fill Unindent * E:NONAM
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][2][2]={1,2,3,4,5,6,7,8},b,r,c;
clrscr();
for(b=0;b<2;b++)
{
printf("%d-Block\n",b);
for(r=0;r<2;r++)
{
for(c=0;c<2;c++) printf("%4d",a[b][r][c]);
printf("\n");
}
}
getch();
}
Watch

```

4:35 PM
06-Dec-22

```
TC
0-Block
  1  2
  3  4
1-Block
  5  6
  7  8
```

4-dimensional array:

An array with several sets, blocks, rows and columns.

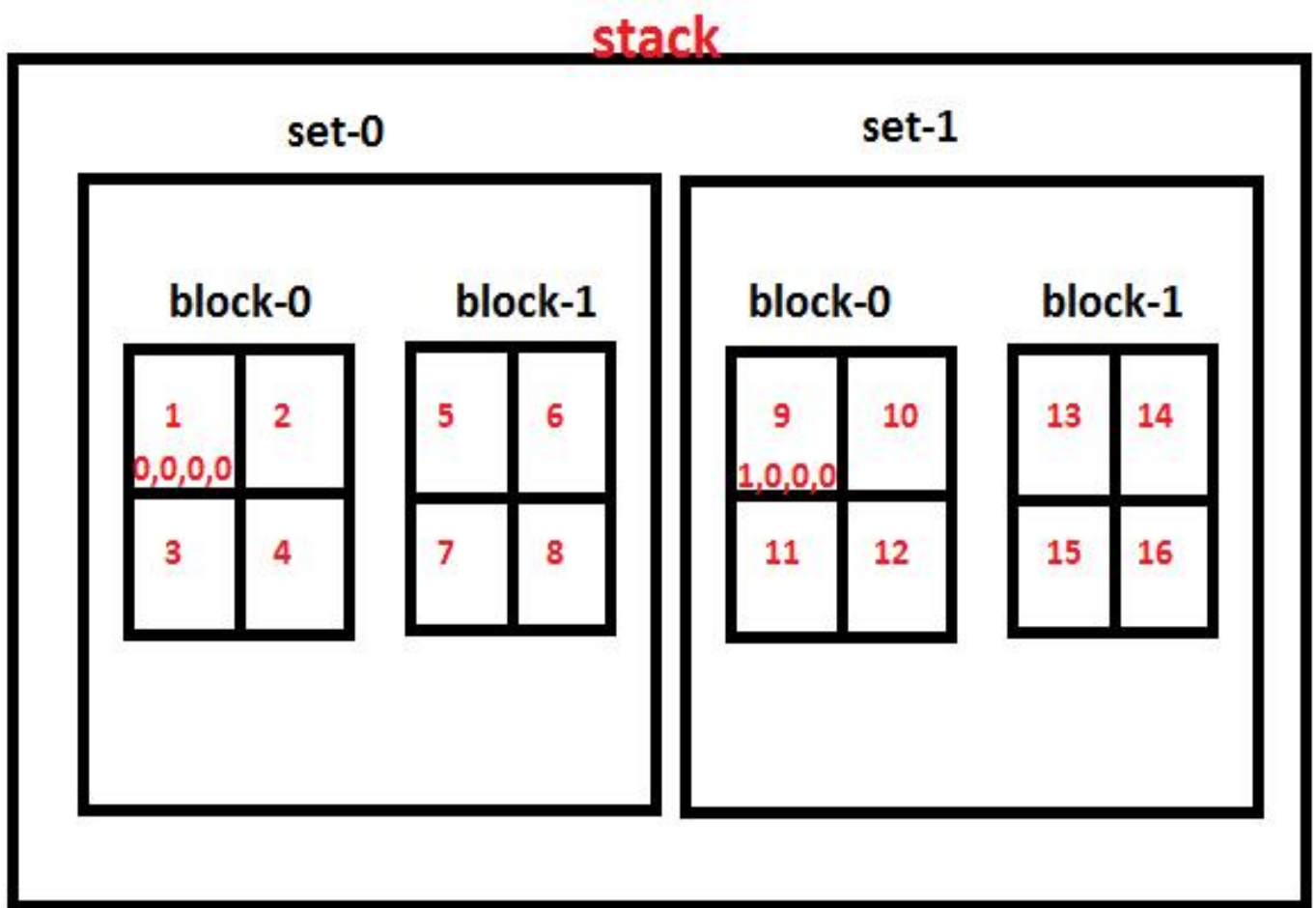
An array with 4 subscripting operators **[] [] [] []**.

Syntax:

**datatype variable [sets] [blocks] [rows]
[cols];**

eg:

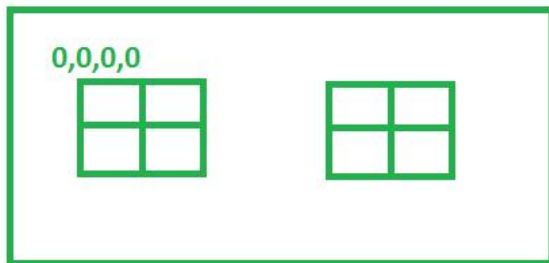
int a[2] [2] [2] [2]= {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};



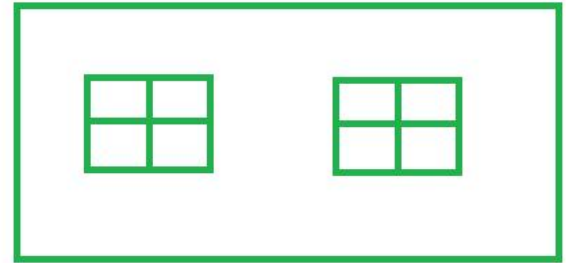

```
datatype var[class][sec][stu][marks];  
int      school[5][2][60][6];
```

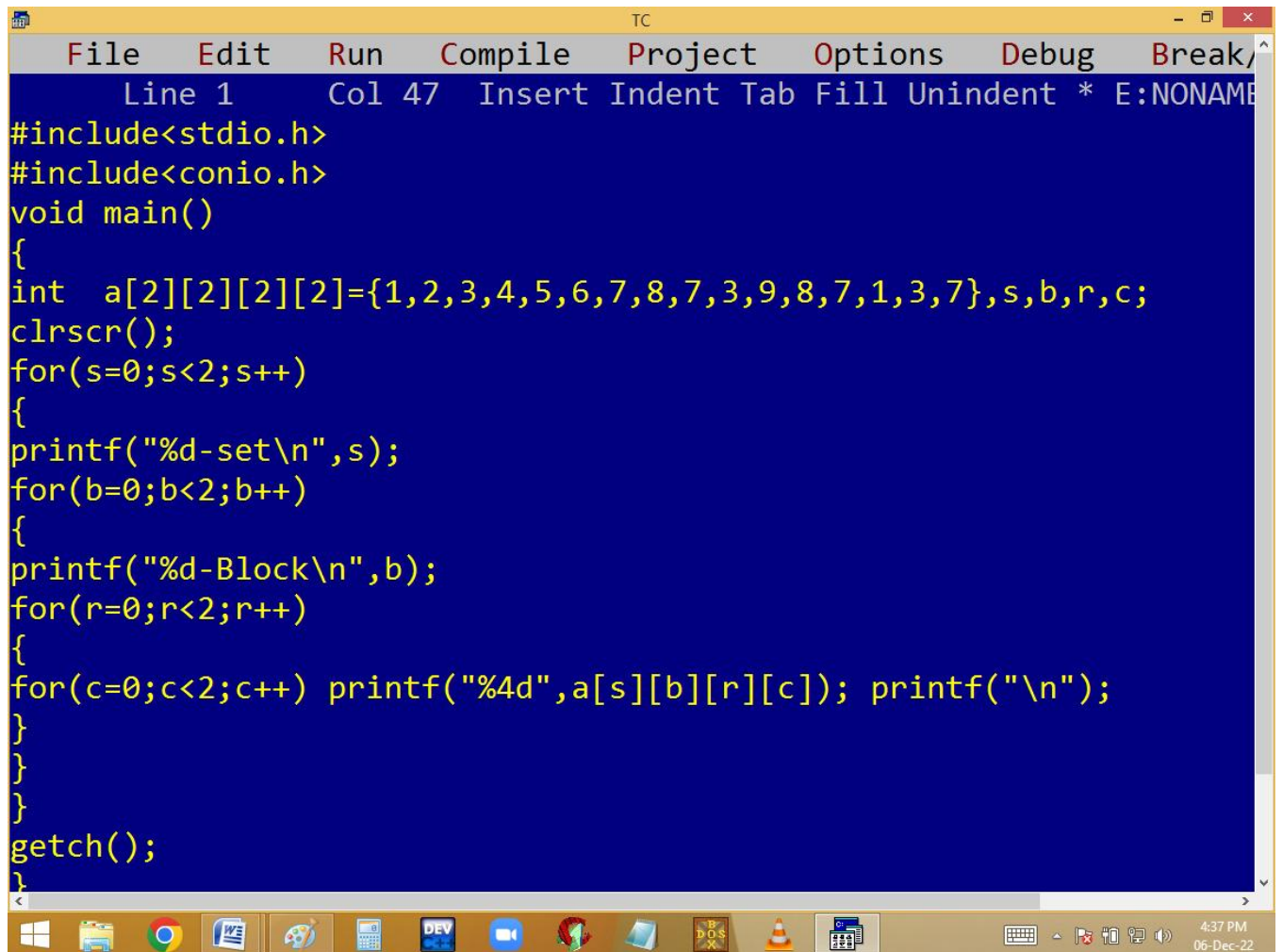
set
int a[2][2][2][2]; an array with 3 subscripts [].
rows
blocks
columns

set-0



set-1





The image shows a screenshot of a Turbo C++ (TC) IDE window. The window has a yellow title bar with the text "TC" and standard window controls. The menu bar includes "File", "Edit", "Run", "Compile", "Project", "Options", "Debug", and "Break". The status bar at the top indicates "Line 1", "Col 47", and "Insert Indent Tab Fill Unindent * E:NONAME". The main editing area has a dark blue background with yellow text. The code is a C program that defines a 2D array 'a' and iterates through it using nested loops. The code is as follows:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int  a[2][2][2][2]={1,2,3,4,5,6,7,8,7,3,9,8,7,1,3,7},s,b,r,c;
clrscr();
for(s=0;s<2;s++)
{
printf("%d-set\n",s);
for(b=0;b<2;b++)
{
printf("%d-Block\n",b);
for(r=0;r<2;r++)
{
for(c=0;c<2;c++) printf("%4d",a[s][b][r][c]); printf("\n");
}
}
}
getch();
}
```

The Windows taskbar is visible at the bottom, showing various application icons and the system clock indicating 4:37 PM on 06-Dec-22.

```
TC
0-set
0-Block
  1  2
  3  4
1-Block
  5  6
  7  8
1-set
0-Block
  7  3
  9  8
1-Block
  7  1
  3  7
```

STRINGS

- A group of characters is called string.
- It is one dimensional character array.
- It is alpha-numeric.
- It is an implicit pointer.
- It is a derived data type.

Note:

- One byte should be left for Null char(**\0**). Otherwise we are getting garbage or junk values. Null char indicates string is completed.
- String variable Size can't be less than string. Otherwise we are getting error.
- Using **=** operator, we can't copy a string into another. We have to use `strcpy()` or copy character by character manually.
- Using **==** (comparison) operator, we can't compare two strings. Use `strcmp()` or compare the characters one by one manually.

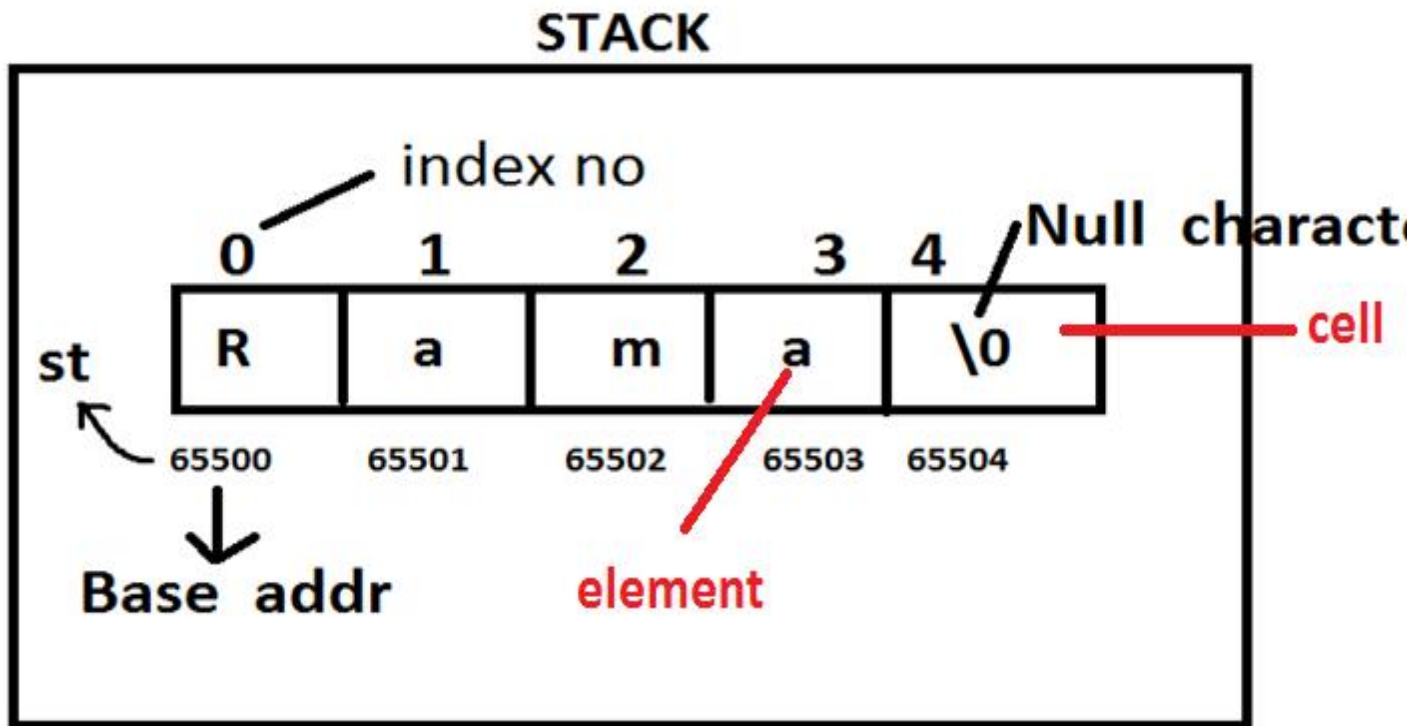
Syntax:

char variable [size] = "string";

or

char variable[]="string";

Eg: `char st[5] = "Rama";`



Note: String is implicit pointer because of string variable stores base address.

String declaration methods:

`char st[5] = "rama";` Ok

`char st[20] = "Naresh It";` Ok

char st [4] = { 'r', 'a', 'm' }; Ok → char array.

char st[3]= "ram"; It gives garbage values in printing.

char st [3] = "rama"; error

char st[0]; error

char st[0]="abc"; Ok

char st[-5]; error

char st[5.5]; error

char st[5%3]; Ok → char st[2];

char st[3+2]; → st[5] → Ok

char st[] ="Ram"; Ok.

char st[] ; error

int n=20;

char st[n]; No

#define n 20

char st[n]; Ok

Note: String variable size always constant positive integer value.